

Harness Optimization is Asynchronous Human-in-the-Loop (HITL) RL

Evaluate agent traces, form a hypothesis, update the harness, and repeat

Challenges in Harness Engineering

1 · Base models keep changing

- A harness improvement is measured against one model version
- As the model evolves, that improvement may shrink or disappear
- Past verification no longer proves that the gain still holds

2 · Final metrics hide the cause

- Long runs contain many traces and engine sessions
- Small behavioral failures accumulate into the final metric
- The metric shows the outcome, but not which behavior caused it

Harness improvement is hard to trust when updates are not reproducible and agent behavior is not observable.

Challenge 1: Model changes invalidate old gains

On the original model

- The harness fixes a repeated model failure
- Benchmarks show a clear improvement over the baseline harness
- The result is valid for that model, harness, and evaluation set

After the model changes

- The base model may fix the failure itself
- The harness change may become redundant or harmful
- The old benchmark result is now stale evidence

A harness gain is always relative to a specific base-model version; model drift creates uncertainty about whether the gain still exists.

Make every harness comparison reproducible

Pin the base model

- Record the exact model version and inference settings
- Self-host the model when long-term version stability is required
- Change one variable at a time: model or harness, not both

Use composable commits

- Put one harness change in each small commit
- Keep its evaluation configuration and results
- Replay, revert, or combine changes on any model version

Reproducibility turns an old result into a test we can run again, instead of a claim we can only remember.

Challenge 2: Final metrics hide small failures

What happens inside a run

- Tool errors, malformed outputs, retries, and timeouts
- Context overflow and unstable multi-step behavior
- Minor failures spread across many traces and engine sessions

What the final metric shows

- It aggregates all of those failures into one number
- It arrives after the full run finishes
- It cannot tell us which behavior should be fixed

The final metric tells us **whether performance changed**; it does not tell us **why it changed**.

Add a behavior-trace observability layer

1 · Collect

- Use one trace schema across engine sessions
- Capture tool calls, errors, retries, context, and outcomes

2 · Evaluate

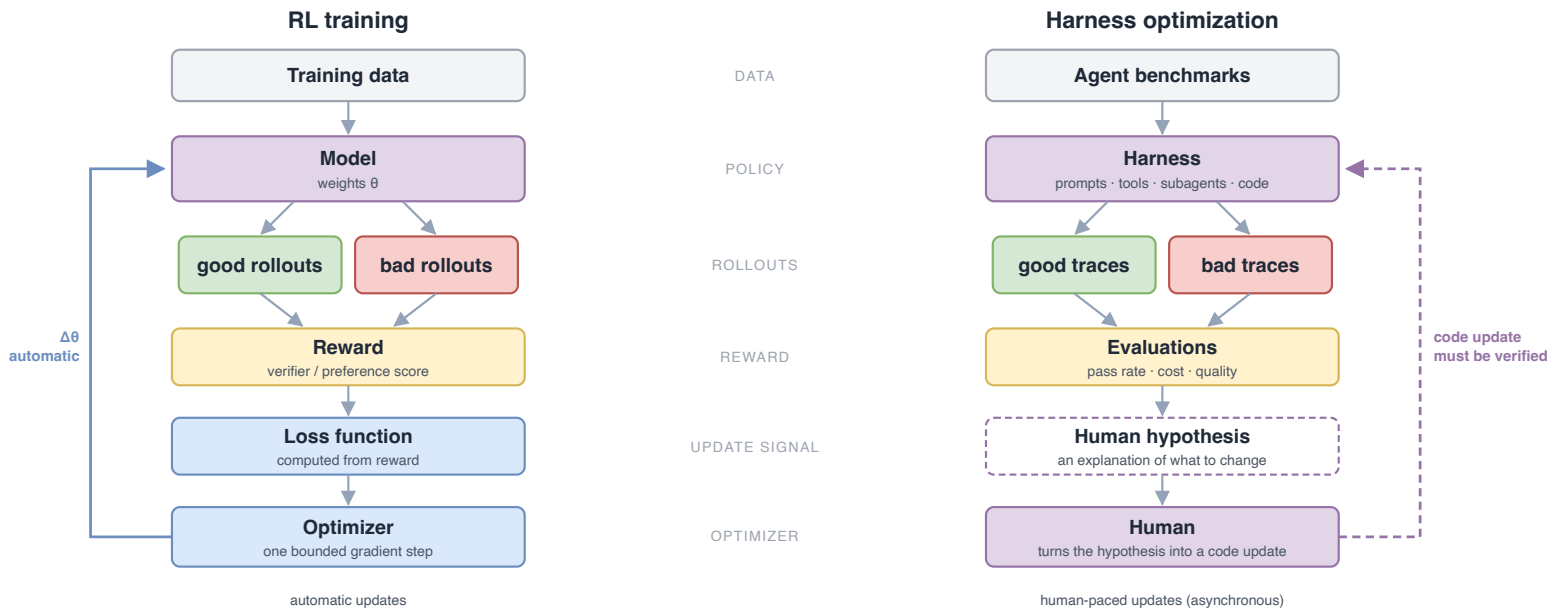
- Run stable checks on specific agent behaviors
- Let an LLM read traces and mine recurring failure patterns

3 · Aggregate

- Measure pattern frequency across runs
- Surface examples before they become a large metric regression

Final metrics evaluate outcomes. Trace-level evaluations explain behavior and point to the next harness change.

Together, these pieces define the engineering loop



Both loops use outcomes from the current system to decide how to update the system next.

The parts of the two loops line up

RL training	Harness optimization
Training data	Agent benchmarks or real tasks
Model	Harness code: prompts, tools, and workflows
Rollouts	Agent traces
Reward	Evaluation scores and qualitative feedback
Loss	A hypothesis about what should change
Optimizer update	A human turns the hypothesis into a code update

The analogy is useful because both systems improve through repeated rollout, evaluation, and update.

Harness optimization has no explicit loss

RL training

- A predefined loss converts reward into an update signal
- The optimizer computes a parameter update automatically
- The same objective is used throughout training

Harness optimization

- Evaluations show what worked and what failed
- A human must explain the failure as a hypothesis
- The hypothesis determines the next code update
- The same hypothesis can guide similar harnesses

An evaluation tells us **how the harness performed**; a hypothesis tells us **what to change**.

Every code update must be evaluated

RL training runs automatically

- Once data and the optimization algorithm are ready, the loop runs automatically
- Each optimizer step is followed by the next rollout
- Updates are small and guided by the loss

Harness updates happen at human speed

- A human reviews traces and updates the code when ready
- A code update can change prompts, tools, or the full workflow
- The update may improve, regress, or simply move the failure

After every code update, run new rollouts and compare the evaluations. Keep the change only when the evidence supports it.

Compare good and bad traces to form hypotheses

What the human does

- Compare successful and failed traces
- Find a repeated failure pattern
- Explain why the harness produces that pattern
- Propose the smallest change that tests the explanation

What a hypothesis agent can do

- Group similar failures across many traces
- Summarize differences between good and bad traces
- Suggest likely causes and candidate changes
- Leave prioritization and final judgment to the human

This part can be partly automated: an agent can mine failure patterns, while the human decides which hypothesis is worth testing.

Three types of harness optimization

1 · Fix incompatibilities

- Repair harness bugs
- Adapt the model to an environment or tool contract
- Recover capability the base model already has

2 · Offload long work

- Move a long subtask behind one tool call
- Use a subagent, skill, or fixed workflow
- Return the result instead of the full trajectory

3 · Give direct guidance

- Encode a solution path you already know
- Add a focused prompt instruction or procedure
- Reduce search when the right approach is known

These changes improve the harness in different ways: fix an interface, shorten the working context, or narrow the search path.

Type 1: Fix model-environment mismatches

Example: inefficient SQL

- A model may omit pushdown aggregation
- The query then fans out and uses too much memory
- If the mistake is occasional, add a prompt reminder
- If it is systematic, enforce a rewrite in code

What this type can and cannot do

- It removes failures at the model-environment boundary
- It usually raises reliability more than reasoning ability
- The fix may be specific to the current base model
- Re-test it whenever the model changes

Compatibility fixes recover blocked capability; they rarely create new reasoning capability in the base model.

Related evidence: VeRO (Ursekar et al., 2026) reports larger gains on tool-use tasks than on reasoning-heavy tasks.

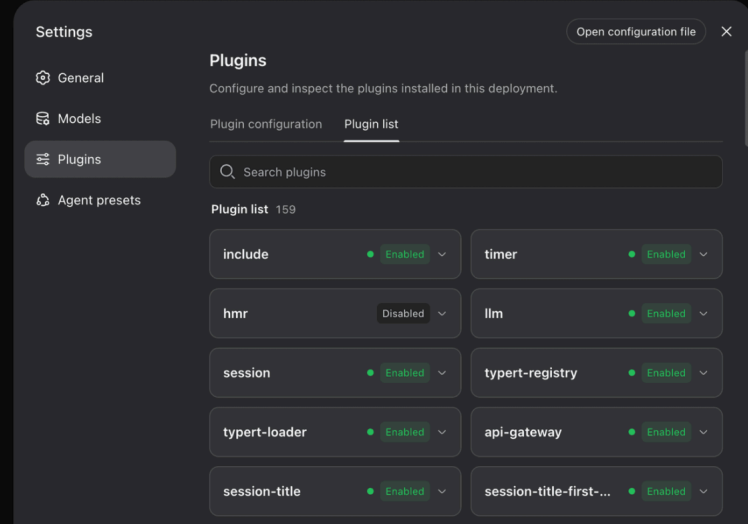
One more thing...

We just found the newly released DeepSeek Harness.
It independently converges on both engineering choices.

Plugins let the harness evolve with the model

Everything is a plugin

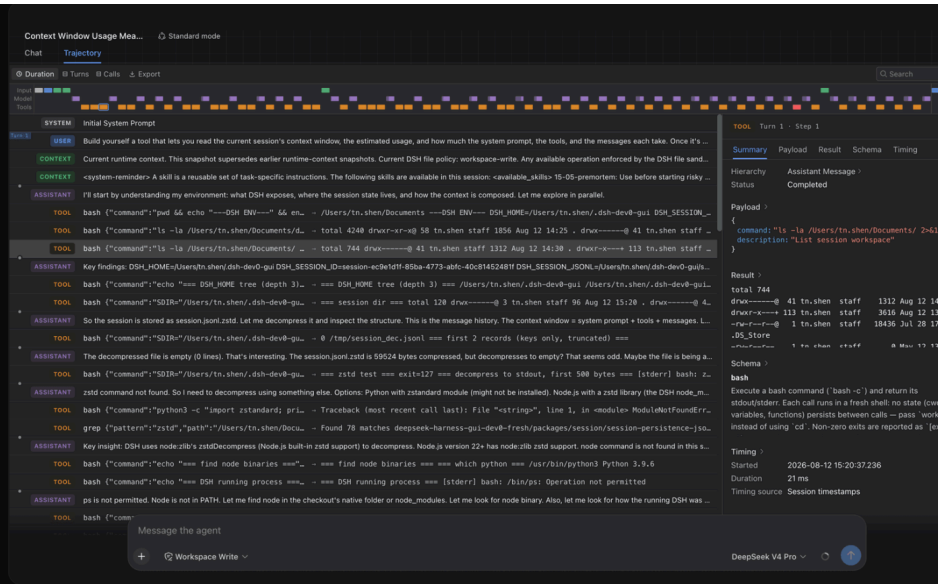
DeepSeek Harness is built on Cordis's plugin system. Plugins provide every agent capability, including models, tools, skills, sessions, sandboxes, storage, loops, scheduling, and the UI. Cordis services and events let the plugins work together. Developers can select, swap, or extend any capability in configuration without changing the DeepSeek Harness source code.



Instead of one fixed harness, DeepSeek composes models, tools, skills, sessions, and loops from swappable plugins. Different model versions can use different harness configurations.

Replayable traces make behavior observable

- ④ Every run is traceable



DeepSeek provides the trace data plane. **The only missing piece is a solid eval suite** that automatically runs full evaluations after every harness update—and provides scientific evidence that the change works.